Customer Churn Prediction Report

Jake Visintin

May 6, 2026

Introduction

I chose to use the Telco Customer Churn data set for this project (IBM Sample Data, n.d.). For this project I was tasked to predict customer churn with the company. My first step was to download the raw data and look at it to see what columns I was working with. Once opening the .csv file in Excel, I formatted the data as a table. I then looked to see if the data had any blanks, to which I realized that the column "TotalCharges" had blanks in it. This would make sense given that a customer might have just joined in the month, so they do not have a total charge logged in the data set. Following this I decided which variables I think would best affect Churn. I noticed that a lot of the variables in the data was in a string format, and the only floats and integers were SeniorCitizen, tenure, MonthlyPayment, and TotalPayment. This worried me given that to use k-nearest neighbors and logistic regression, these were going to be put in float/integer fashion. The X variables that I decided to test for Churn (y) were Contract, Monthly Charges, Total Charges, and Tenure.

Coding

I first began by loading the data into the code by using a link to the raw data contained in my GitHub repo. I then started converting all blanks in the total charge's column into 0. This was necessary as blanks are seen as strings. I decided to convert them rather than just scrape the blanks, because like previously said, a customer might have just joined in the month, so they do not have a total charge logged in the data set. I then converted all the variables that I was using from string into a float form. These variables include contract and churn.

I started first by only trying the contract variable and running it through a K-Nearest Neighbors (KNN) model with no scaler. For the n_neighbors, I decided 83 because there are 7,043 customers in the raw data, and this number square rooted is 83.9, and I chose to round down to an odd number. I wanted to use this as a sort of baseline before I added more future variables, and I got a score of 73.46%.[1] This is important because as I add more variables to X, it should give me a better idea of if the variables adding do contribute towards predicting churn.

Next, I chose to test variables that were floats to begin to predict churn, which were monthly charges and total charges. The score I got for this was 79.58%.[1] I then decided to test tenure

[1] The accuracy score obtained was adapted from Schwab-McCoy (n.d.) section 2.1

given that tenure shows how long a customer has been with the company. After testing monthly charges, total charges, and tenure to predict churn, I got a score of 79.54%.[1] Since the score went down, this made me think that tenure was not a good variable to test to predict churn. So, knowing this, I tested monthly charges, total charges, and contract, to which I got a score of 79.89%.[1] which was the highest recorded so far. Taking this into account I decided to train and test these exact variables using an 80/20 split. This gave me an accuracy score of 80.77%. [2]

Figure 1 Code explaining analysis

```
# The logic for this was adapted from
# zyBook "Machine Learning" by Aimee Schwab-McCoy, Section 10.2

# Input and output features
X = df[["MonthlyCharges", "TotalCharges", "Contract"]]
y = df[["Churn"]]

# train/test split (80/20)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Scale
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Initialize and fit
knn = KNeighborsClassifier(n_neighbors=83)
knn.fit(X_train_scaled, np.ravel(y_train))

# Predict
knnpred = knn.predict(X_test_scaled)

# Accuracy
score = knn.score(X_test_scaled, y_test)
print(f"KNN Accuracy: {score:.2%}")
```

```
KNN Accuracy: 80.77%
```

Note. Image created by author. See Appendix for the link to view the Python code and data used to create this figure.

Seeing that these variables gave me the highest accuracy score, I decided to compare the KNN model to a logistics regression model. For the first test I used monthly charges, total charges, and contract as the x, and churn as the y. This in turn gave me an accuracy score of 78.66%.[3] I then chose to also test these variables with the addition of tenure. This gave me a higher accuracy score of 78.83%.[3]

[2] The test and train method was adapted from Schwab-McCoy (n.d.) section 10.2
[3] The accuracy score using a logistic regression model was adapted from Schwab-McCoy (n.d.) section 2.2

Figure 2 Code explaining analysis

```
# The logic for this was adapted from
# zyBook "Machine Learning" by Aimee Schwab-McCoy, Section 2.2

# Input and output features
X = df[["MonthlyCharges", "TotalCharges", "tenure", "Contract"]]
y = df[["Churn"]]

# Initialize and fit
logisticModel = LogisticRegression(penalty=None)
logisticModel.fit(X, np.ravel(y))

# Predict
lrmpred = logisticModel.predict(X)

#Proprotion of instances correctly classified
lr_score = logisticModel.score(X, np.ravel(y))
print(f"LR Accuracy: {lr_score:.2%}")
```

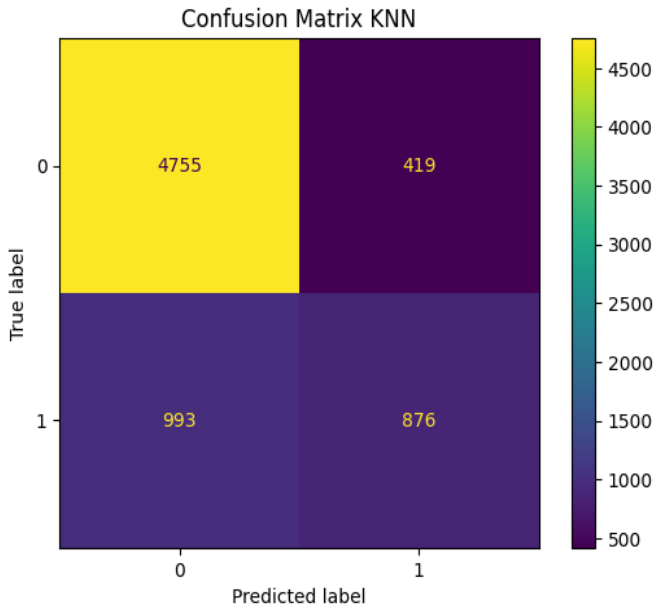
```
LR Accuracy: 78.83%
```

Note. Image created by author. See Appendix for the link to view the Python code and data used to create this figure.

When comparing the two models, the KNN model gave me the highest accuracy score. Now I decided that it would be best to see and compare the models on a confusion matrix. For the confusion matrix KNN, I used the variables that got me the highest score, monthly charges, total charges, and contract for my x, and churn as my y. For the Confusion Matrix LR, I used the variables that got me the highest score which were monthly charges, total charges, tenure, and contract. [4]

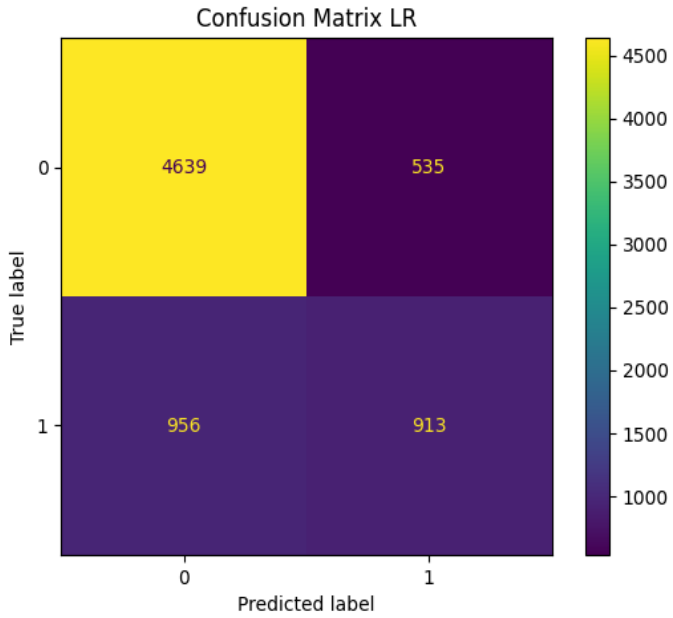
[4] The confusion matrix used was adapted from Schwab-McCoy (n.d.) section 4.2 and 4.5

Figure 3 Confusion Matrix KNN



Note. Image created by author. See Appendix for the link to view the Python code and data used to create this figure.

Figure 4 Confusion Matrix LR



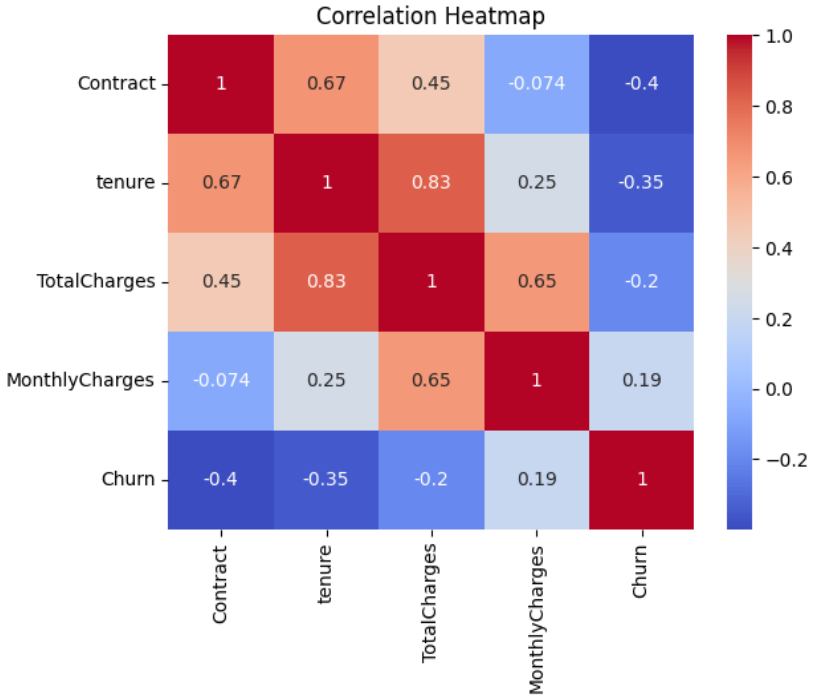
Note. Image created by author. See Appendix for the link to view the Python code and data used to create this figure.

When comparing the two, the KNN confusion matrix has more true positives than the LR matrix. This makes sense given that the KNN model outperformed the LR model in accuracy score.

Correlation Heatmap

When looking at the correlation heatmap, it can be inferred that churn and monthly changes seem to increase together, whereas tenure and churn appear to decrease together. If monthly charges were to increase, then there would be an increase in churn. This is true in a real-world scenario, because if the price of something was to increase then the demand would decrease, leading to higher churn. The opposite can be said to be tenure. This makes logical sense seeing that as a customer is with the company for longer, they are deemed more loyal, and less likely to break contracts, decreasing churn.

Figure 5 Correlation Heatmap



Note. Image created by author. See Appendix for the link to view the Python code and data used to create this figure.

Recommendations

I recommend that the KNN model be selected as a predictor for churn. Seeing that the accuracy score for the KNN model is higher than the LR model, leads me to believe that the KNN model is a better choice. I also recommend that the Monthly Charges (MonthlyCharges), Total Charges

(TotalCharges), and Contract variables be considered when wanting to predict churn within this company given that they showed the highest accuracy.

Appendix

GitHub Repo

All work done in python code and raw data used for this analysis can be found on my GitHub repository: https://github.com/toplr/customer_churn_prediction

References

IBM Sample Data Sets. (n.d.). *Telco customer churn* [Data set]. Kaggle. https://www.kaggle.com/datasets/blastchar/telco-customer-churn

Schwab-McCoy, A. (n.d.). *Machine Learning*. zyBooks. zybooks.com ISBN 979-8-203-15456-9.